PostgreSQL и подключение базы данных к Node.js-приложению

Переходим от временного массива products[] к постоянному хранению данных

Результаты занятия

- ✓ Объяснять роль PostgreSQL в серверном приложении
- ✓ Понимать клиент-серверную модель PostgreSQL
- ✓ Создавать базу данных и таблицу products
- ✓ Подключать пакет pg к Node.js-приложению
- ✓ Настраивать параметры подключения через .env
- ✓ Использовать Pool для выполнения SQL-запросов
- ✓ Получать строки PostgreSQL в JavaScript
- ✓ Подключать базу данных к слою Service

Учебные вопросы

1	**Зачем приложению PostgreSQL**	Постоянное хранение данных и роль реляционной базы данных
2	**Подготовка базы данных**	База, пользователь, таблица products и тестовые данные
3	**Подключение Node.js к PostgreSQL**	Пакет pg, параметры подключения, Pool и проверка соединения
4	**SQL-запросы из Node.js**	pool.query(), result.rows и параметризованные запросы
5	**PostgreSQL в архитектуре Express-приложения**	Route → Controller → Service → Pool → PostgreSQL

Учебный вопрос 1. Зачем приложению PostgreSQL

Данные должны переживать перезапуск Node.js-приложения

Что происходило с products[] раньше

- Товары хранились в обычном JavaScript-массиве.
- Service выполнял find(), push(), Object.assign() и splice().
- Такой вариант удобен для изучения REST API и архитектуры приложения.
- Массив существует только в оперативной памяти процесса Node.js.
- После остановки сервера все добавленные и изменённые данные теряются.

Главная идея

Массив подходит для учебного прототипа, но не является постоянным хранилищем данных.

Массив в памяти и база данных

JavaScript-массив

- Хранится в памяти процесса
- Исчезает после перезапуска
- Удобен для простых демонстраций
- Нет полноценного языка запросов
- Плохо подходит для совместной работы многих пользователей

PostgreSQL

- Хранит данные постоянно
- Данные сохраняются после перезапуска Node.js
- Поддерживает SQL
- Обеспечивает ограничения целостности
- Предназначена для одновременной работы клиентов

Что такое реляционная база данных

- Реляционная база данных хранит данные в связанных таблицах, состоящих из строк и столбцов.
- Таблица описывает набор однотипных сущностей.
- Строка представляет один объект.
- Столбец представляет свойство объекта.
- Первичный ключ однозначно идентифицирует строку.
- SQL используется для создания, чтения и изменения данных.

От прототипа к реальному приложению!

От массива *products*[] к постоянному хранилищу

Те же данные.
Больше возможностей.

Node.js + массив в памяти

Данные хранятся в оперативной памяти (RAM)

Node.js
(процесс)

```
// Массив в памяти
let products = [
  { id: 1, name: "Ноутбук", price: 50000 },
  { id: 2, name: "Мышь", price: 1500 },
  { id: 3, name: "Клавиатура", price: 3000 }
];
```

Оперативная память (RAM)
Данные существуют только пока работает процесс Node.js

Перезапуск сервера



Сервер остановлен (Node.js завершен) → Перезапуск → Сервер запущен снова

После перезапуска сервера данные в памяти сбрасываются.
Массив products[] возвращается к начальному состоянию.
Все изменения пользователей потеряны!

Переход от временного хранения к постоянному

- Вместо хранения в RAM — сохраняем в базу данных
- Используем PostgreSQL
- Данные остаются даже после перезапуска

Надёжные приложения строятся на надёжных хранилищах!

PostgreSQL + таблица products

Данные хранятся в постоянном хранилище (на диске)



PostgreSQL
(база данных)

Таблица products

id	name	price
1	Ноутбук	50000
2	Мышь	1500
3	Клавиатура	3000
4	Монитор	25000

Постоянное хранилище (на диске)
Данные сохраняются независимо от работы приложений

Перезапуск приложения Node.js



Сервер остановлен (Node.js завершен) → Перезапуск → Сервер запущен снова

Строки таблицы остаются без изменений!

Данные в PostgreSQL сохраняются, даже если Node.js перезапустить десятки раз.

JavaScript-массив удобен для прототипа, но PostgreSQL обеспечивает постоянное хранение данных.

Сегодня прототип — завтра реальный проект!

Основные элементы реляционной модели

Database

База данных содержит таблицы и другие объекты приложения.

Table

Таблица хранит набор однотипных сущностей, например товары.

Row

Строка таблицы представляет один конкретный объект.

Column

Столбец описывает отдельное свойство объекта.

Primary Key

Первичный ключ уникально идентифицирует каждую строку.

Что такое PostgreSQL

- PostgreSQL — реляционная система управления базами данных, с которой приложение взаимодействует через SQL и сетевое соединение.
- PostgreSQL работает как отдельный серверный процесс.
- База данных не находится внутри Node.js-приложения.
- Node.js выступает клиентом PostgreSQL.
- Приложение отправляет SQL-запросы серверу базы данных.
- PostgreSQL выполняет запрос и возвращает результат.

Клиент-серверная модель PostgreSQL

- PostgreSQL Server принимает подключения клиентов.
- Клиентом может быть psql, pgAdmin, Node.js-приложение или другая программа.
- Клиент отправляет SQL-запрос.
- PostgreSQL анализирует и выполняет запрос.
- Результат возвращается клиентскому приложению.
- Клиент и сервер базы данных могут находиться на разных компьютерах.

Одно приложение — много возможностей!

Node.js-приложение как клиент PostgreSQL

Разные клиенты — один сервер данных!

Node.js-приложение
Клиентское приложение

node Express

```
const express = require('express');
const { Pool } = require('pg');

// Отправляем SQL-запрос к PostgreSQL
const result = await pool.query(
  'SELECT * FROM users');
console.log(result.rows);
```

- Формирует SQL-запросы
- Обрабатывает полученные данные (rows) в коде приложения

1 **SQL-запрос**

Например: SELECT, INSERT, UPDATE, DELETE

2 **Строки таблиц / результат запроса**

PostgreSQL возвращает данные в виде строк (rows)

Клиент отправляет запрос — сервер возвращает данные!

Сервер PostgreSQL
Отдельный сервер базы данных

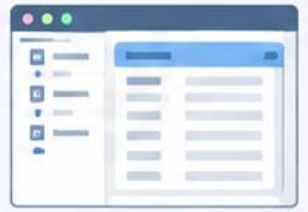
PostgreSQL

- Ожидает подключения от клиентов
- Выполняет SQL-запросы
- Хранит данные в таблицах
- Обеспечивает безопасность и управление доступом

Другие клиенты PostgreSQL
Также могут подключаться к тому же серверу

pgAdmin
Графический интерфейс для работы с PostgreSQL

pgAdmin



psql
Командная строка для работы с PostgreSQL

psql

```
psql -U postgres -d mydb
mydb=# SELECT * FROM users;
 id  | name
-----+--------
  1  | Анна
  2  | Иван
(2 rows)
```

PostgreSQL работает как отдельный сервер, а Node.js-приложение подключается к нему как клиент.

Тот же сервер, разные инструменты — одни данные!

Таблица products в PostgreSQL

Столбец	Пример	Назначение
id	15	Уникальный идентификатор товара
name	SSD 1TB	Название товара
price	45000	Цена товара
category	Components	Категория товара
in_stock	true	Признак наличия

Учебный вопрос 2. Подготовка базы данных

Создаём отдельную базу shop_db для Internet Shop API

Что требуется для подключения к PostgreSQL

Host

Адрес компьютера, на котором работает PostgreSQL. Локально обычно localhost.

Port

Сетевой порт PostgreSQL. Стандартное значение — 5432.

Database

Имя базы данных, к которой подключается приложение.

User

Пользователь PostgreSQL, от имени которого выполняются запросы.

Password

Пароль пользователя базы данных.

Пять параметров подключения к PostgreSQL

Node.js может подключиться к PostgreSQL, только если все параметры указаны правильно и совпадают с настройками сервера.

База данных делает идеи реальностью

1. host = localhost

Адрес сервера PostgreSQL (например, localhost или IP-адрес).

2. port = 5432

Сетевой порт, на котором работает PostgreSQL (по умолчанию 5432).

3. database = shop_db

Имя базы данных, к которой подключаемся (например, shop_db).

Node.js-приложение

node.js + Express

```
const { Pool } = require('pg');
// Подключение к PostgreSQL
```

Подключение

Все пять параметров передаются при подключении к базе данных.

PostgreSQL



Сервер PostgreSQL с базой данных **shop_db**

Правильные настройки = успешное подключение!

4. user = shop_user

Пользователь PostgreSQL, от имени которого выполняется подключение.

5. password = ******

Пароль пользователя PostgreSQL.

PostgreSQL надежная основа ваших проектов

Все пять параметров должны совпадать с настройками PostgreSQL.

Проверьте и поключайтесь!

Создаём пользователя и базу данных

```
CREATE USER shop_user
WITH PASSWORD 'college123';

CREATE DATABASE shop_db
OWNER shop_user;
```

Что важно

- shop_user — отдельная учётная запись приложения.
- Пароль используется при подключении Node.js.
- shop_db — база данных интернет-магазина.
- OWNER shop_user делает созданного пользователя владельцем базы.
- В реальном проекте пароль должен быть значительно сложнее учебного.

Создаём таблицу products

```
CREATE TABLE products (
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  price INTEGER NOT NULL CHECK (price > 0),
  category VARCHAR(100) NOT NULL,
  in_stock BOOLEAN NOT NULL DEFAULT true
);
```

Что важно

- id создаётся автоматически и является первичным ключом.
- name хранит название товара.
- price хранится как целое число тенге.
- CHECK запрещает цену меньше или равную нулю.
- in_stock хранит логическое значение наличия товара.

Добавляем тестовые данные

```
INSERT INTO products
  (name, price, category, in_stock)
VALUES
  ('Keyboard', 15000, 'Accessories', true),
  ('Mouse', 8000, 'Accessories', true),
  ('Monitor', 85000, 'Monitors', false);
```

Проверяем таблицу

Запрос: SELECT * FROM products ORDER BY id;

```
1 | Keyboard | 15000 | Accessories | true
2 | Mouse    | 8000  | Accessories | true
3 | Monitor  | 85000 | Monitors    | false
```

Пояснение

- До подключения Node.js необходимо убедиться, что база, таблица и тестовые данные действительно существуют.

Учебный вопрос 3. Подключение Node.js к PostgreSQL

Используем драйвер node-postgres и пул соединений

Зачем Node.js нужен драйвер

- Драйвер базы данных — библиотека, позволяющая приложению устанавливать соединение с СУБД, отправлять запросы и получать результаты.
- Express сам по себе не умеет работать с PostgreSQL.
- Для PostgreSQL в Node.js часто используется пакет pg.
- Пакет реализует PostgreSQL-протокол.
- Он поддерживает Promise и async/await.
- Через него приложение выполняет SQL-запросы.

Устанавливаем node-postgres

```
npm install pg
```

Client и Pool

- Client представляет одно соединение с PostgreSQL.
- Pool управляет набором повторно используемых соединений.
- Веб-приложение одновременно обслуживает несколько HTTP-запросов.
- Для серверного Express-приложения обычно удобнее использовать Pool.
- Один Pool обычно создаётся при запуске приложения и используется повторно.

Client и Pool

Client

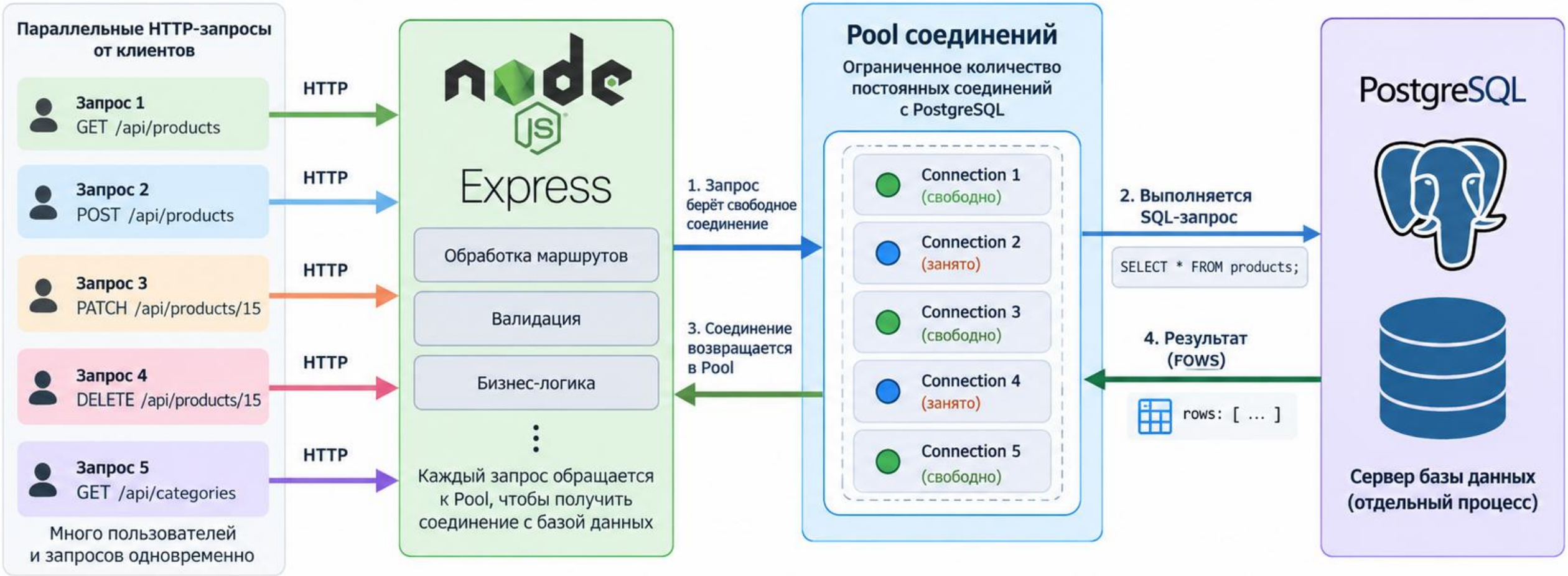
- Одно соединение
- Нужно явно подключать
- Нужно явно закрывать
- Удобен для небольшого отдельного скрипта
- Не лучший вариант для каждого HTTP-запроса

Pool

- Управляет несколькими соединениями
- Повторно использует подключения
- Удобен для веб-сервера
- pool.query() автоматически получает соединение
- Один Pool можно использовать во всех services

Слайд №26. Зачем Express-приложению Pool соединений

Pool соединений позволяет обрабатывать много параллельных запросов, используя ограниченное количество соединений с PostgreSQL, вместо создания нового соединения для каждого запроса.



Преимущества Pool

- Переиспользование соединений
- Быстрая обработка запросов
- Ограниченное количество соединений
- Меньшая нагрузка на PostgreSQL

Важно

Не создавайте новое соединение для каждого HTTP-запроса! Используйте Pool (например, из пакета pg), чтобы эффективно работать с PostgreSQL.

Пример настройки

```
const pool = new Pool({
  host: 'localhost',
  port: 5432,
  database: 'shop_db',
  user: 'shop_user',
  password: '******',
  max: 10 // максимальное количество соединений
});
```

Параметры подключения в .env

```
PGHOST=localhost
PGPORT=5432
PGDATABASE=shop_db
PGUSER=shop_user
PGPASSWORD=college123
```

Почему параметры подключения не записывают в код

- Пароль базы данных не должен находиться внутри исходного JavaScript-файла.
- На разных компьютерах адрес базы и пароль могут отличаться.
- Переменные окружения позволяют менять конфигурацию без изменения кода.
- Файл .env не следует публиковать в Git-репозитории.
- Для локальной разработки .env позволяет хранить отдельную конфигурацию проекта.

Главная идея

Код приложения и секреты подключения должны храниться раздельно.

Создаём config/db.js

```
import { loadEnvFile } from 'node:process';
import { Pool } from 'pg';

loadEnvFile();

export const pool = new Pool({
  host: process.env.PGHOST,
  port: Number(process.env.PGPORT),
  database: process.env.PGDATABASE,
  user: process.env.PGUSER,
  password: process.env.PGPASSWORD
});
```

Что важно

- loadEnvFile() загружает переменные из файла .env.
- Pool импортируется из пакета pg.
- process.env содержит параметры подключения.
- Создаётся один общий объект pool.
- Services смогут импортировать pool и выполнять SQL-запросы.

Проверяем соединение

```
const result = await pool.query(
  'SELECT NOW() AS current_time'
);

console.log(result.rows[0]);
```

Что важно

- pool.query() отправляет SQL-запрос PostgreSQL.
- SELECT NOW() не зависит от таблицы products.
- Успешный ответ подтверждает, что соединение работает.
- Строки результата находятся в result.rows.

Что происходит при pool.query()

1	**Node.js вызывает pool.query()**	Приложение передаёт SQL-запрос объекту Pool.
2	**Pool получает соединение**	Используется свободное соединение или создаётся новое в пределах настроек пула.
3	**SQL отправляется PostgreSQL**	Запрос передаётся серверу базы данных.
4	**PostgreSQL выполняет запрос**	СУБД обрабатывает SQL и формирует результат.
5	**Результат возвращается Node.js**	Приложение получает объект результата с массивом rows.

Типичные ошибки подключения

Проблема	Что проверить
ECONNREFUSED	Запущен ли PostgreSQL и правильно ли указан host/port
password authentication failed	PGUSER и PGPASSWORD
database does not exist	PGDATABASE и наличие базы
permission denied	Права пользователя на базу и таблицу
relation products does not exist	Создана ли таблица products в нужной базе

Учебный вопрос 4. SQL-запросы из Node.js

Service начинает получать товары из PostgreSQL

Что возвращает pool.query()

- pool.query() возвращает Promise.
- После await мы получаем объект результата запроса.
- result.rows содержит строки результата SELECT.
- Каждая строка представлена обычным JavaScript-объектом.
- result.rowCount содержит количество строк, затронутых или возвращённых запросом.
- Service может преобразовать result.rows в данные REST API.

Получаем все товары из PostgreSQL

```
export async function getAllProducts() {
  const result = await pool.query(
    `SELECT
       id,
       name,
       price,
       category,
       in_stock AS "inStock"
     FROM products
     ORDER BY id`
  );

  return result.rows;
}
```

Что важно

- Service становится асинхронным, потому что запрос к базе требует ожидания.
- SELECT получает данные из таблицы products.
- SQL-поле in_stock преобразуется в API-поле inStock через alias.
- result.rows возвращается controller.
- JavaScript-массив products[] больше не является источником данных.

GET /api/products теперь читает PostgreSQL

Запрос: GET /api/products

```
SELECT id, name, price, category,
       in_stock AS "inStock"
FROM products
ORDER BY id;
```

Результат • JSON


```
[
  {
    "id": 1,
    "name": "Keyboard",
    "price": 15000,
    "category": "Accessories",
    "inStock": true
  }
```


Пояснение

- REST API для клиента остаётся прежним, но источником данных становится PostgreSQL.

Получаем товар по id

```
export async function findProductById(id) {
  const result = await pool.query(
    `SELECT
       id,
       name,
       price,
       category,
       in_stock AS "inStock"
     FROM products
     WHERE id = $1`,
    [id]
  );

  return result.rows[0];
}
```

Что важно

- $1 обозначает параметр SQL-запроса.
- Значение параметра передаётся отдельно в массиве [id].
- Драйвер не объединяет значение id со строкой SQL вручную.
- result.rows[0] содержит найденный объект.
- Если строка не найдена, результат будет undefined.

Параметризованные SQL-запросы

- Данные пользователя не следует вставлять в SQL через конкатенацию строк.
- PostgreSQL-параметры обозначаются как $1, $2, $3 и далее.
- Значения передаются отдельным массивом.
- Так код отделяет SQL-команду от пользовательских данных.
- Параметризованные запросы являются базовой защитой от SQL injection.

Главная идея

Используйте WHERE id = $1 и [id], а не формируйте SQL из частей пользовательского ввода.

Слайд №39. Как req.params.id превращается в параметр $1

Значение из URL передаётся в запрос к PostgreSQL безопасно — через параметризованный запрос.

Безопасный код — надёжное приложение!

1 HTTP-запрос от клиента

Пользователь запрашивает товар с id = 15

```
http://localhost:3000/api/products/15

GET  /api/products/15
```



Клиент
(браузер, frontend)

2 Controller получает параметр

Express распределяет запрос и передаёт параметр в контроллер

productController.js

```
export async function
getProductById(req, res, next) {
  const idParam = req.params.id;
  console.log(idParam); // "15"
  // дальше преобразуем в число
}
```

`req.params.id = "15"`

Тип: строка (string)

3 Преобразование к числу

Контроллер преобразует строку в число перед передачей в Service

productController.js

```
const idParam = req.params.id;
const id = Number(idParam);

console.log(id); // 15
```

`id = 15`

Тип: число (number)

4 Service вызывает параметризованный запрос

Service передаёт значение как параметр $1 (массив значений)

productService.js

```
export async function
findProductById(id) {
  const result = await pool.query(
  `SELECT id, name, price, category,
          in_stock AS "inStock"
     FROM products
     WHERE id = $1`,
   [id]
  );
  return result.rows[0];
}
```

`pool.query("... WHERE id = $1", [15])`

Значение 15 передаётся отдельно от SQL-команды

5 PostgreSQL выполняет запрос

PostgreSQL получает запрос с параметром, выполняет его и возвращает строку



PostgreSQL

id	name	price	category	in_stock
15	Laptop	42000	Laptops	true

Небезопасный вариант (не используйте)

Конкатенация строки с пользовательским вводом приводит к SQL injection.

productService.js

```
// ПЛОХО! Не делайте так
const sql = "SELECT * FROM products WHERE id = " + id;
const result = await pool.query(sql);
```

Проблема

- Пользовательский ввод напрямую вставляется в SQL.
- Возможны SQL injection атаки.
- Приложение становится уязвимым.
- Нельзя гарантировать тип и формат данных.

Безопасный вариант (используйте)

- Параметры $1, $2, ... отделяют SQL от данных.
- Драйвер **pg** правильно экранирует значения.
- Защита от SQL injection.
- Корректная работа с типами данных.
- Рекомендуемый подход для всех запросов.

Параметризованные запросы — стандарт профессиональной разработки!

Controller ожидает результат Service

```
export async function getProducts(req, res, next) {
  try {
    const products = await getAllProducts();
    res.json(products);
  } catch (error) {
    next(error);
  }
}
```

Что важно

- Controller становится async.
- await ожидает завершения SQL-запроса в Service.
- Успешный результат отправляется клиенту как JSON.
- Ошибка PostgreSQL передаётся централизованному errorHandler через next(error).

Controller получения одного товара

```
export async function getProductById(req, res, next) {
  try {
    const id = Number(req.params.id);
    const product = await findProductById(id);

    if (!product) {
      return next(
        new AppError('Product not found', 404)
      );
    }

    res.json(product);
  } catch (error) {
    next(error);
  }
}
```

Что важно

- id по-прежнему приходит из req.params.
- Controller не содержит SQL.
- Service выполняет запрос к PostgreSQL.
- Отсутствующий товар по-прежнему приводит к 404.
- Архитектура HTTP-слоя почти не изменилась.

Учебный вопрос 5. PostgreSQL в архитектуре Express-приложения

Меняется источник данных, но ответственность слоёв сохраняется

Service до и после подключения PostgreSQL

Раньше: массив

- products.find()
- products.push()
- Object.assign()
- products.splice()
- Данные хранятся в памяти Node.js

Теперь: PostgreSQL

- SELECT
- INSERT
- UPDATE
- DELETE
- Данные постоянно хранятся в базе

Слайд №44. Route → Controller → Service → Pool → PostgreSQL

Каждый слой выполняет свою задачу. SQL-запросы находятся в Service, а Route и Controller с ними не работают.

Чистая архитектура — поддерживаемый код!

1 HTTP Request

Запрос от клиента (браузер, frontend)

`GET /api/products/15`



Клиент

2 Product Router

Выбирает нужный обработчик (controller) по маршруту

productRoutes.js

```
router.get(
  '/:id',
  productController.
  getProductById
);
```

Только маршрутизация. Не содержит SQL.

3 Product Controller

Получает параметры, валидирует и вызывает service

productController.js

```
export async function
getProductById(req, res, next) {
  const id = Number(req.params.id);
  const product = await
    productService.findById(id);
  if (!product) {
    return res.status(404).json({
      message: 'Product not found'
    });
  }
  res.json(product);
```

Работает с HTTP (req, res, коды ответов). Не содержит SQL.

4 Product Service

Содержит бизнес-логику и выполняет SQL-запросы через Pool

productService.js

```
export async function findById(id) {
  const result = await pool.query(
    `SELECT id, name, price, category,
            in_stock AS "inStock"
     FROM products
     WHERE id = $1`,
    [id]
  );
  return result.rows[0];
}
```

Здесь выполняются SQL-запросы. Работает с данными, а не с HTTP.

5 pg Pool

Управляет набором соединений с PostgreSQL

Pool соединений

- Connection 1
- Connection 2
- Connection 3
- ⋮
- Connection N

Предоставляет соединение для выполнения запроса и возвращает его в Pool.

6 PostgreSQL

Выполняет SQL-запрос и возвращает данные



PostgreSQL



Таблица products

rows (результат запроса)

id	name	price	category	in_stock
15	Laptop	42000	Laptops	true

Возврат данных в Service (result.rows)

Service возвращает данные в Controller (product)

Controller формирует JSON-ответ (res.json(product))

JSON Response

```
{
  "id": 15,
  "name": "Laptop",
  "price": 42000,
  "category": "Laptops",
  "inStock": true
}
```

Разделение ответственности

- **Route** — только маршрутизирует запросы.
- **Controller** — работает с HTTP и вызывает Service.
- **Service** — содержит бизнес-логику и SQL-запросы.
- **Pool** — управляет соединениями.
- **PostgreSQL** — хранит данные и выполняет SQL.

Важно помнить

- SQL-запросы находятся в Service.
- Route и Controller не должны содержать SQL.
- Такое разделение упрощает поддержку и тестирование.
- Можно менять источник данных (например, с массива на PostgreSQL), не изменяя публичный REST API.

Пример полного пути запроса

1. Клиент отправляет `GET /api/products/15`
2. Router передаёт запрос в `productController.getProductById()`
3. Controller получает `id` и вызывает `productService.findById(id)`
4. Service выполняет SQL через `pool.query()`
5. PostgreSQL возвращает строки (`rows`)
6. Данные возвращаются обратно и отправляются клиенту как JSON

Полный путь GET /api/products/2

1	**Route выбирает Controller**	GET /api/products/:id передаётся функции getProductById.
2	**Controller получает id**	Значение req.params.id преобразуется в число.
3	**Controller вызывает Service**	findProductById(2) вызывается с обычным JavaScript-значением.
4	**Service выполняет pool.query()**	PostgreSQL получает SELECT ... WHERE id = $1 с параметром [2].
5	**PostgreSQL возвращает строку**	Service получает результат в result.rows[0].
6	**Controller формирует HTTP-ответ**	Найденный товар отправляется клиенту как JSON, а отсутствие товара приводит к 404.

Что изменилось в архитектуре

- Routes практически не изменились.
- Controllers по-прежнему работают с req, res и HTTP-кодами.
- Services теперь являются асинхронными и выполняют SQL.
- Появился отдельный модуль config/db.js с Pool.
- PostgreSQL стал постоянным источником данных.
- Frontend продолжает использовать те же REST endpoints.

Главная идея

Правильное разделение слоёв позволяет заменить источник данных без изменения публичного REST API.

Мини-практика

Подключите Internet Shop API к PostgreSQL и замените чтение товаров из массива операциями SELECT.

Задание

- Создать базу shop_db
- Создать таблицу products
- Добавить несколько тестовых товаров
- Установить пакет pg
- Создать .env с параметрами PostgreSQL
- Создать config/db.js и объект Pool
- Проверить соединение через SELECT NOW()
- Переписать getAllProducts() с использованием SELECT
- Переписать findProductById(id) с использованием WHERE id = $1

Критерии проверки

- Node.js успешно подключается к PostgreSQL
- GET /api/products получает данные из таблицы
- GET /api/products/:id выполняет параметризованный запрос
- Перезапуск Node.js не удаляет товары
- SQL находится в Service, а не в Route или Controller

Контрольные вопросы

1. 1. Почему JavaScript-массив не подходит для постоянного хранения данных?
2. 2. Что такое PostgreSQL и какую роль она выполняет в приложении?
3. 3. Какие параметры необходимы для подключения к PostgreSQL?
4. 4. Для чего Node.js-приложению нужен пакет pg?
5. 5. Чем Pool отличается от одного Client?
6. 6. Почему пароль базы данных следует хранить в переменных окружения?
7. 7. Где находятся строки результата после pool.query()?
8. 8. Почему Service после подключения базы становится асинхронным?
9. 9. Для чего используются параметры $1, $2 и массив значений?
10. 10. Какие слои приложения должны знать о SQL?

Итоги занятия

1. PostgreSQL обеспечивает постоянное хранение данных серверного приложения.
2. Node.js подключается к PostgreSQL как клиент.
3. Для взаимодействия с PostgreSQL используется драйвер pg.
4. В веб-приложении удобно использовать Pool соединений.
5. Параметры подключения хранятся отдельно от исходного кода.
6. pool.query() позволяет отправлять SQL и получать result.rows.
7. Пользовательские значения передаются в SQL через параметры $1, $2 и далее.
8. SQL-запросы размещаются в Service, сохраняя разделение Route → Controller → Service.
9. Публичный REST API может остаться неизменным после перехода от массива к PostgreSQL.